Football Analytics API

XJCO3011 Web Services and Web Data
Coursework 1

Chen Haotian

April 2026

Project Overview & Objectives

CORE MISSION

- Develop a robust, data-driven RESTful web service for Premier League football statistics.
- Bridge the gap between raw sports data and programmable, analytical access.
- Demonstrate advanced API design principles, relational data modeling, and secure authentication.

KEY CAPABILITIES

- Full CRUD management for Teams, Players, and Matches.
- Complex data aggregation for dynamic league tables and rankings.

20

API ENDPOINTS

3

CORE DATA MODELS

45

AUTOMATED TESTS

System Architecture & Technology Stack

Python 3.11

CORE LOGIC

Chosen for its unparalleled ecosystem in data processing and readability. Native async support enables non-blocking I/O.

FastAPI

PRESENTATION

Delivers high performance (Starlette) and strict type safety (Pydantic). Auto-generates interactive Swagger UI docs.

SQLAlchemy

DATA ACCESS

Abstracts complex SQL into Pythonic objects. Enforces relational integrity and mitigates SQL injection vulnerabilities.

SQLite

STORAGE

Provides zero-configuration portability. Fully supports standard SQL operations required for demonstrating relational modeling.

Relational Database Schema Design

Team

The central entity storing club information including name, city, stadium, manager, and budget.

- 1–to–Many relationship with Players
- 1–to–Many relationship with Matches (Home)
- 1–to–Many relationship with Matches (Away)

Player

Stores individual statistics such as goals, assists, appearances, and disciplinary cards.

- Linked to exactly one Team
- Uses team_id as Foreign Key
- Establishes a clear hierarchy

Match

Captures the outcome and detailed statistics of a game, including possession and shots.

- Contains two Foreign Keys
- home_team_id points to Team
- away_team_id points to Team

Core API Functionality: CRUD Operations

RESOURCE	READ (PUBLIC)	WRITE (SECURED)	QUERY CAPABILITIES
Teams	GET /teams/ GET /teams/{id}	POST, PUT, DELETE	Filter by city Search by name
Players	GET /players/ GET /players/{id}	POST, PUT, DELETE	Filter by team, position, nationality Sort by goals, assists, cards
Matches	GET /matches/ GET /matches/{id}	POST, PUT, DELETE	Filter by season, team Filter by date range

Universal Pagination
page & page_size parameters on all lists

Strict Validation
Pydantic schemas enforce data integrity

Standardized Responses
Consistent JSON structures & HTTP codes

Advanced Analytics Endpoints

League Table

GET /analytics/league-table

Calculates standings from match results (points, GD, W/D/L).

Team Performance

GET /analytics/team-performance/{id}

Team metrics including home/away form and scoring averages.

Player Rankings

GET /analytics/player-rankings

Top players sorted by goals, assists, or other stats.

Head-to-Head

GET /analytics/head-to-head

Historical comparisons and match outcomes between two teams.

Season Summary

GET /analytics/season-summary

Overview of the season: total and average goals per match.

These endpoints show complex data aggregation, transforming raw data into high-level football intelligence.

Security: JWT Authentication

STATELESS MECHANISM

JSON Web Tokens (JWT)

Implemented using python-jose. Provides a highly scalable, stateless authentication mechanism. The server does not need to maintain session state in memory or a database, reducing overhead.

Secure Password Hashing

Passwords are never stored in plaintext. They are securely hashed using bcrypt before being persisted to the database, ensuring protection against data breaches.

ROUTE PROTECTION STRATEGY

Public Endpoints

All GET requests (retrieving teams, players, matches, and analytics) are freely accessible without authentication, allowing public data consumption.

Protected Endpoints

All write operations (POST, PUT, DELETE) require a valid JWT token passed in the Authorization header.

Client Request	→	Includes Bearer Token
FastAPI Middleware	→	Validates Signature & Expiry
Route Handler	✓	Executes DB Operation

Testing Methodology & Coverage

RIGOROUS VALIDATION STRATEGY

Complete Test Isolation

Utilizes a dedicated in-memory SQLite database (sqlite:///:memory:). Tables are freshly created and dropped for every single test to guarantee deterministic results.

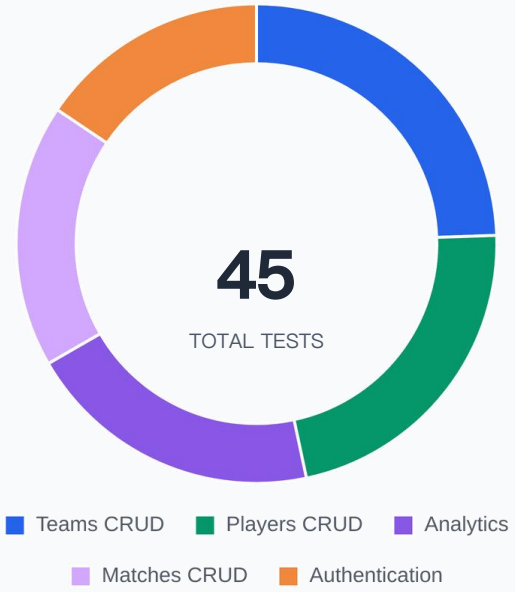
Pytest Fixture Injection

Heavy reliance on fixtures to streamline setup. Fixtures automatically inject database sessions, TestClients, seed data, and valid JWT headers.

Explicit Edge Case Coverage

Tests explicitly validate HTTP status codes (200, 201, 400, 401, 403, 404, 422) and handle edge cases like invalid foreign keys and duplicate registrations.

45-TEST SUITE BREAKDOWN



Version Control Practices & Commit History

Atomic & Frequent Commits

Code changes are committed in small, logical units (e.g., "改进API验证与认证", "解决文件相对路径报错问题"). This ensures easier tracking, debugging, and reverting if necessary.

Descriptive Commit Messages

Every commit message clearly explains **what** was changed and **why**, providing a transparent history of the development process for all collaborators.

Structured Repository

The GitHub repository maintains a clean structure , alongside essential files like app , tests , docs , and data with separated directories for .gitignore and requirements.txt .

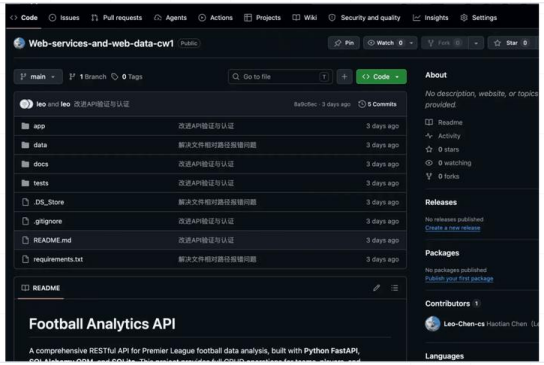


Fig 1: Repository Structure & Recent Commits

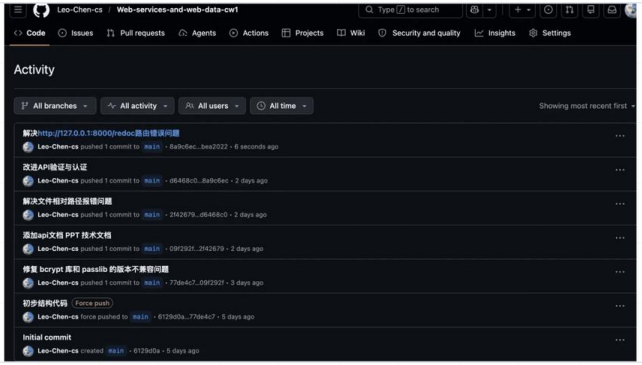


Fig 2: Detailed Commit Activity Log

API Documentation Overview

The API Documentation is meticulously authored using LaTeX and delivered as a professional PDF document. It serves as the definitive contract for consumers integrating with the Football Analytics API.

Comprehensive Coverage

- Details all 20 RESTful endpoints across Teams, Players, Matches, Analytics, and Auth.
- Clearly defines required and optional query parameters and path variables.
- Explains the JWT Authentication flow and header requirements.

Developer Ready

- Provides exact example requests for immediate testing.
- Illustrates expected JSON responses for both success and failure scenarios.
- Documents standard HTTP error codes (400, 401, 404, 500).

Academic Formatting

- Typeset in Times New Roman to meet strict coursework guidelines.
- Structured with clear hierarchical headings and monospaced code blocks.
- Maintained directly alongside the codebase in the /docs directory.

Technical Report Highlights

Architecture & Design

- Detailed justification for selecting FastAPI (asynchronous performance, automatic Swagger UI) and SQLite (zero-configuration portability).
- Explanation of the layered architecture separating routing, business logic, and data access.
- Analysis of the relational database schema, including the handling of dual foreign keys for matches.

GenAI Declaration

- Transparent disclosure of AI assistance in code scaffolding and documentation formatting.
- Includes complete Conversation Logs as an appendix, adhering strictly to the 5-page limit for the main content.

Testing Methodology

- Comprehensive overview of the Pytest suite covering 45 independent test cases.
- Explanation of testing strategies: happy paths, edge cases, authentication failures, and data validation.

Challenges & Learnings

- Reflection on resolving SQLAlchemy AmbiguousForeignKeysError during model relationships setup.
- Documentation of debugging the bcrypt version compatibility issue that caused initial test failures.

All Deliverables Summary

Code Repository

Complete FastAPI source code, SQLAlchemy models, SQLite database, and Pytest suite. Maintained on GitHub with a structured commit history demonstrating iterative development.

✓ READY

API Documentation

A comprehensive, LaTeX-typeset PDF documenting all 20 endpoints. Includes detailed request/response formats, authentication flows, and HTTP status codes.

✓ READY

Technical Report

A 5-page academic report (Times New Roman) justifying architectural choices, detailing testing methodologies, and including a complete GenAI usage declaration log.

✓ READY